AI WITH DURGA SIR
ONE INPUT, THREE AI APPLICATIONS
BUILD YOUR OWN AI CONTENT STUDIO

60-MINUTE WORKSHOP | STUDENT NOTES

Sunday, 18 October 2026 | 7:00 PM IST

Generative AI • Python • LangChain • LCEL

LEARNING CONCLUSION "One idea can become many opportunities when you know how to build the right system."

1. What will you build?

Enter just one topic, such as "Benefits of Learning Python". Your program creates three useful outputs:

1. Blog Outline — structured points for an article.
2. Social Media Post — an attractive LinkedIn post.
3. Marketing Email — an email with a subject and call to action.

MASTER APPLICATION FLOW

```
ONE TOPIC
    |
    v
AI CONTENT STUDIO
    |
    +--> Blog Outline
    +--> Social Media Post
    +--> Marketing Email
```

LEARNING CONCLUSION "One input does not mean only one output."

2. Six fundamentals everyone must know

Concept	Very simple meaning
Generative AI	AI that generates new content from instructions.
LLM	A Large Language Model that works with language.
Prompt	The input or instruction given to an AI model.
Chat Model	An interface that uses messages to interact with an AI model.
LangChain	A framework to connect components for LLM applications.
Runnable	A component with standard execution methods such as invoke().

LEARNING CONCLUSION "The model provides intelligence. The developer decides how to use it."

3. First working AI application

A direct model call follows: Prompt → Chat Model → AIMessage → response.content.

PROGRAM 1 — DIRECT MODEL CALL

```
import os
from getpass import getpass
from langchain_openai import ChatOpenAI
```

```
key = os.getenv("OPENAI_API_KEY")
if not key:
    key = getpass("OpenAI API key: ").strip()
if not key:
    raise ValueError("API key required")

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=key
)
response = model.invoke(
    "Write 5 benefits of learning Python."
)
print(response.content)
```

Key point: model.invoke() generally returns an AIMessage. Its content contains the generated answer text.

LEARNING CONCLUSION "Using AI is a skill. Building AI applications is another skill."

4. PromptTemplate: Make prompts reusable

Static prompt: "Explain Python." The topic is fixed.

Dynamic prompt: "Explain {topic}." The topic changes without changing the complete program.

```
Template: Explain {topic}.
Input:    topic = Python
Output:   Explain Python.
```

PromptTemplate creates reusable prompts with variables such as {topic}.

LEARNING CONCLUSION **"A reusable program can solve many similar problems without rewriting the whole code."**

5. Output Parser and LCEL

A Chat Model generally returns an AIMessage. StrOutputParser gives us the generated plain text as a Python string.

```
PromptTemplate
      |
      v
ChatOpenAI
      |
      v
StrOutputParser
      |
      v
String
```

LCEL stands for LangChain Expression Language. It connects compatible components using the pipe operator (|).

THE LCEL HERO LINE

```
chain = prompt | model | parser
result = chain.invoke(data)
```

The output of the left component becomes the input to the right component.

LEARNING CONCLUSION **"First understand the components. Then understand the data flow. Finally connect them."**

6. Complete LCEL mini-program

PROGRAM 2 — PROMPT + MODEL + PARSER

```
import os
from getpass import getpass
from langchain_openai import ChatOpenAI
from langchain_core.prompts import (
    PromptTemplate
)
from langchain_core.output_parsers import (
    StrOutputParser
)

key = os.getenv("OPENAI_API_KEY")
if not key:
    key = getpass("OpenAI API key: ").strip()
if not key:
    raise ValueError("API key required")

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=key
)
prompt = PromptTemplate.from_template(
    "Give 5 simple points about {topic}."
)
parser = StrOutputParser()
chain = prompt | model | parser

topic = input("Enter topic: ").strip()
if not topic:
    topic = "Python"
result = chain.invoke({"topic": topic})
print(result)
```

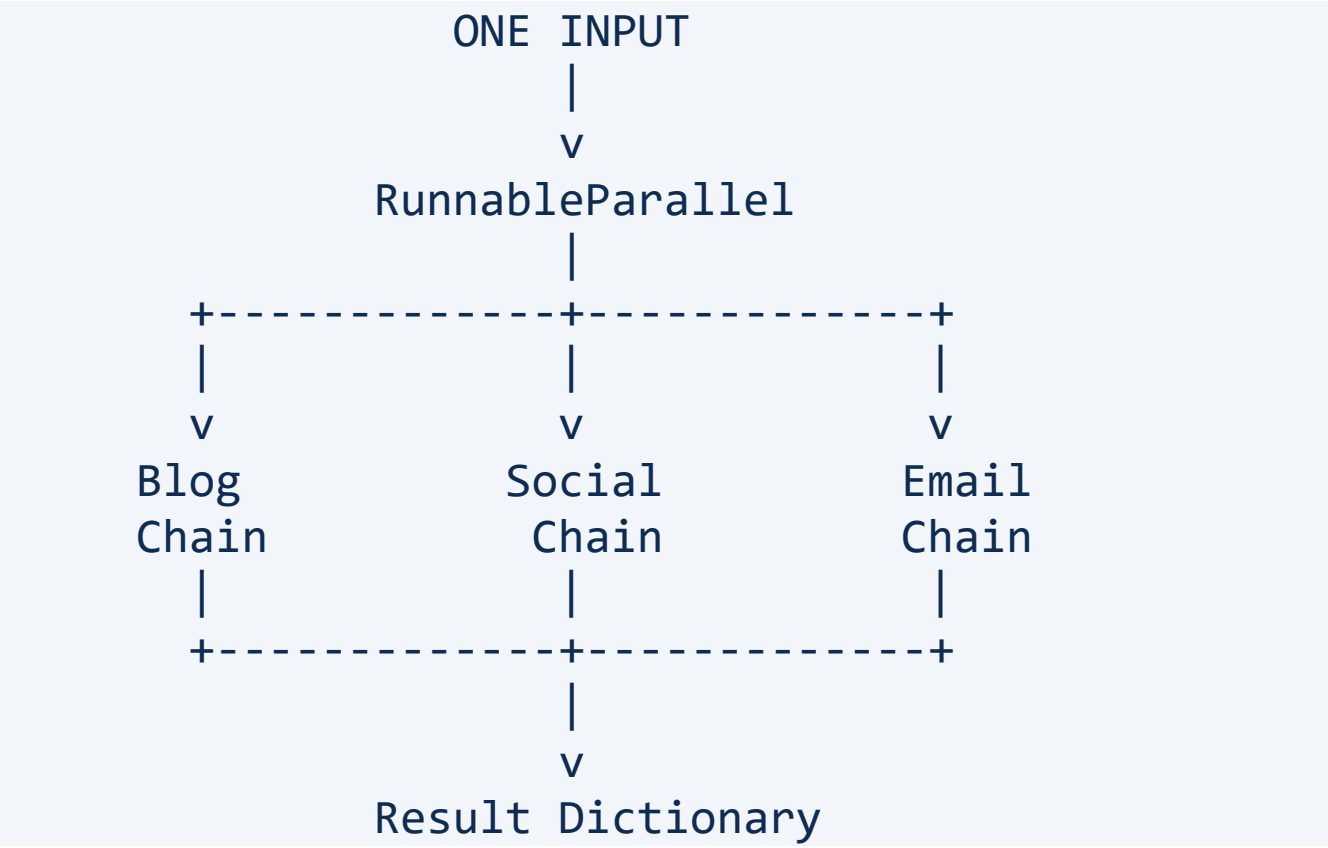
7. Today's most important concept

RunnableParallel — One input, many branches

Suppose our input is “Benefits of Learning Python”. We want three different outputs. The three tasks do not depend on each other.

RunnableParallel organises independent Runnable branches and can execute them concurrently. It returns their results in a dictionary.

PARALLEL ARCHITECTURE



LEARNING CONCLUSION **“When tasks depend on each other, use sequence. When tasks are independent, consider parallel.”**

8. RunnableSequence vs RunnableParallel

RunnableSequence: Runs dependent steps one after another. Each next step receives the output from the previous step. Example: Prompt → Model → Parser.

RunnableParallel: Runs independent branches concurrently. Each branch receives the same input. Example: Blog + Social + Email.

Remember: parallel execution can reduce waiting time, but speed depends on network, API and rate limits.

9. Practice RunnableParallel without AI

This example uses only Python functions. It does not need an API key.

PROGRAM 3 — NO AI KEY REQUIRED

```
from langchain_core.runnables import (
    RunnableLambda,
    RunnableParallel
)

def heading(data):
    return "Learning " + data["topic"]

def hashtag(data):
    return "#" + data["topic"].replace(
        " ", ""
    )

def count(data):
    return len(data["topic"].split())

parallel = RunnableParallel(
```

```
    heading=RunnableLambda(heading),
    hashtag=RunnableLambda(hashtag),
    words=RunnableLambda(count)
)
result = parallel.invoke(
    {"topic": "Generative AI"}
)
print(result)
```

EXPECTED RESULT

```
{
    'heading': 'Learning Generative AI',
    'hashtag': '#GenerativeAI',
    'words': 2
}
```

There is one input dictionary, three Runnable branches and one result dictionary.

LEARNING CONCLUSION "A well-designed system can perform many independent tasks from one request."

10. The final AI Content Studio

Our final application has three LCEL chains. Every chain has its own PromptTemplate, but all three reuse the same model and string output parser.

MASTER CODE — THREE AI BRANCHES

```
blog_chain = blog_prompt | model | parser
social_chain = social_prompt | model | parser
email_chain = email_prompt | model | parser

studio = RunnableParallel(
```

```
    blog_outline=blog_chain,
    social_post=social_chain,
    marketing_email=email_chain
)
```

One invocation runs the combined workflow:

```
result = studio.invoke({"topic": topic})
```

result is a dictionary. Access values using keys: blog_outline, social_post and marketing_email.

LEARNING CONCLUSION "The real power of LangChain is how we connect and coordinate reusable components."

11. Final complete executable project

Install the required packages one time:

INSTALLATION

```
python -m pip install --upgrade langchain-openai
langchain-core
```

Requirements: Python 3.10+, internet connection and a valid OpenAI API key with available API credits. Save as ai_content_studio.py and run using python ai_content_studio.py.

PROGRAM 4 — COMPLETE AI CONTENT STUDIO

```
import os
from getpass import getpass
from pathlib import Path

from langchain_openai import ChatOpenAI
from langchain_core.prompts import (
    PromptTemplate
```

```
)
from langchain_core.output_parsers import (
    StrOutputParser
)
from langchain_core.runnables import (
    RunnableParallel
)

# 1. API key and model
key = os.getenv("OPENAI_API_KEY")
if not key:
    key = getpass("OpenAI API key: ").strip()
if not key:
    raise ValueError("API key required")

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=key
)
parser = StrOutputParser()

# 2. Three separate prompts
blog_prompt = PromptTemplate.from_template(
    """Prepare a blog outline about {topic}.
    Give a title, short introduction,
    5 headings and a short conclusion.
    Use simple English."""
)

social_prompt = PromptTemplate.from_template(
    """Write a LinkedIn post about {topic}.
    Start with an attractive opening.
    Give 3 points, one call to action
    and 3 relevant hashtags.
    Keep it within 100 words."""
```

```
)

email_prompt = PromptTemplate.from_template(
    """Write a marketing email about {topic}.
    Give a subject, friendly greeting,
    3 benefits and a call to action.
    Use simple English. Do not invent
    dates, prices or offers."""
)

# 3. Three LCEL chains
blog_chain = blog_prompt | model | parser
social_chain = social_prompt | model | parser
email_chain = email_prompt | model | parser

# 4. Parallel workflow
studio = RunnableParallel(
    blog_outline=blog_chain,
    social_post=social_chain,
    marketing_email=email_chain
)

# 5. One input
print("\nAI CONTENT STUDIO")
topic = input("Enter a topic: ").strip()
if not topic:
    topic = "Benefits of Learning Python"

# 6. Run the three branches
print("Generating content. Please wait...")
result = studio.invoke({"topic": topic})

# 7. Display results
for key_name, title in [
    ("blog_outline", "BLOG OUTLINE"),
```

```
    ("social_post", "SOCIAL MEDIA POST"),
    ("marketing_email", "MARKETING EMAIL")
]:
    print("\n" + "=" * 30)
    print(title)
    print("=" * 30)
    print(result[key_name])

# 8. Save results as text files
folder = Path("AI_Content_Studio_Outputs")
folder.mkdir(exist_ok=True)

for key_name in result:
    filename = folder / (key_name + ".txt")
    filename.write_text(
        result[key_name],
        encoding="utf-8"
    )

print("\nAll three files saved in:")
print(folder.resolve())
```

12. Understand the result

Suppose the input topic is “Benefits of Learning Python”. The generated text will vary from run to run.

```
result = {
    "blog_outline": "...",
    "social_post": "...",
    "marketing_email": "..."
}
```

Our application also creates the folder AI_Content_Studio_Outputs with three text files:

- blog_outline.txt
- social_post.txt
- marketing_email.txt

Important: one RunnableParallel.invoke() starts three separate model requests. API charges may apply.

LEARNING CONCLUSION "An AI response becomes more useful when our application knows how to organise and use it."

13. Common student doubts

Q. Do we need three ChatOpenAI objects?
A. No. The program reuses one model object across three separate model requests.

Q. Can we ask for all three outputs in one prompt?
A. Yes. RunnableParallel is useful when separate branches, prompts or processing steps are required.

Q. Does RunnableParallel always run faster?
A. Not always. Actual time depends on network, limits and model processing.

Q. Is RunnableParallel the same as batch()?
A. No. batch() processes multiple inputs for one Runnable; RunnableParallel has multiple branches for one input.

Q. Are AI-generated outputs always correct?
A. No. Always review facts, spelling and claims before publishing.

14. Three quick MCQs

1. What does RunnableParallel mainly do?

- A. Trains an LLM
- B. Runs independent Runnable branches
- C. Stores conversation history

Answer: B. It organises multiple independent Runnable branches.

2. What is the result type of our studio?

- A. Dictionary
- B. Database
- C. Only one string

Answer: A. Every branch result is available under a named key.

3. Why do we use three separate prompts?

- A. Three API keys are needed
- B. Each task requires different instructions
- C. Python needs three files

Answer: B. Different content formats need different prompt instructions.

15. Practice assignment

Use the final program and try the following three topics:

1. Benefits of Learning Generative AI
2. Importance of Communication Skills
3. How Time Management Helps Students

Challenge: modify the social media branch into an Instagram caption generator. Keep the other two branches unchanged.

LEARNING CONCLUSION "Learn one concept deeply. Then apply it to different real-world problems."

16. Five final learning points

1. PromptTemplate prepares reusable instructions.
2. ChatOpenAI uses an AI model to generate a response.
3. StrOutputParser returns generated text as a Python string.
4. LCEL connects compatible components using the pipe operator.
5. RunnableParallel runs independent branches and collects their results.

LEARNING CONCLUSION "Do not just use AI to get answers. Learn how to build systems that solve problems."